

Industrial Digital Twin

Comprehensive MATLAB Framework Documentation

Personal Project Documentation

September 15, 2026

Abstract

This document serves as the comprehensive, detailed documentation for a custom-built Industrial Digital Twin framework developed in MATLAB. It covers every class, script, mathematical model, and test suite implemented throughout the project lifecycle. The framework employs Object-Oriented Programming (OOP) to rigorously simulate physical processes, electrical systems, and instrumentation layers, while providing SCADA integration capabilities via a native tag-based historian architecture.

Contents

1	Introduction & Architectural Philosophy	3
1.1	Project Purpose	3
1.2	Core Architectural Principles	3
2	Core Framework Library	4
2.1	IndustrialObject.m	4
2.2	Equipment.m	4
3	Process Equipment Library	5
3.1	ETank.m (Cylindrical Tank)	5
3.2	EPipe.m (Pipeline)	5
3.3	EPump.m (Centrifugal Pump)	5
3.4	Valves: EValve.m & EControlValve.m	5
3.5	Advanced Units: EMixer, EHeater, EFilter	6
4	Electrical & Actuation Library	7
4.1	EMotor.m & EVFD.m	7
4.2	EConveyor.m	7
5	Instrumentation Layer	8
5.1	ETransmitter.m (Base Sensor)	8
6	SCADA & System Architecture	9
6.1	ETagDatabase.m (Plant Server)	9
6.2	EHistorian.m (Time-Series Logger)	9
6.3	EFactory.m & ESimulationEngine.m	9
7	Testing and Validation	10
7.1	Test Coverage	10
8	Examples & Use Cases	11
8.1	TankExample.m	11
8.2	PumpTankExample.m (Early Phase)	11
8.3	HistorianExample.m	12
8.4	InstrumentationExample.m (Mid Phase)	13
8.5	FullFactoryExample.m (Final Phase)	14
9	User Manual: How to Build Your Own Plant	16
9.1	Step 1: Instantiating the Environment	16
9.2	Step 2: Declaring Equipment	16
9.3	Step 3: Attaching Instrumentation	16
9.4	Step 4: Registration	16

9.5	Step 5: Running the Simulation	17
9.6	Step 6: Extracting Data	17
10	Conclusion	18

Chapter 1

Introduction & Architectural Philosophy

1.1 Project Purpose

The goal of this project is to provide a robust simulation environment capable of acting as a true “Digital Twin” for an industrial plant. Traditional MATLAB scripts often mix physical calculations with control logic and data logging, creating monolithic, hard-to-maintain codebases. This framework adopts strict Object-Oriented Programming (OOP) principles to isolate physical processes, sensor measurement errors, and SCADA data aggregation.

1.2 Core Architectural Principles

1. **Reference Semantics (handle classes):** Every component in the plant is passed by reference. This ensures that when a pump reads from a tank, they are interacting with the exact same memory instance, guaranteeing mass and energy conservation across the simulated plant.
2. **Physical Truth vs. Measured Output:** The internal **State** of a device represents perfect physics. The **Outputs** represent what a PLC would see. This separation allows the injection of realistic sensor noise without breaking the laws of thermodynamics.
3. **Standardized Tag Output:** Every piece of equipment implements a `getTags()` method, outputting a standard industrial tag format (Name, Value, Unit, Quality, Timestamp) ready for OPC UA or SQL Historians.

Chapter 2

Core Framework Library

Located in `library/core/`, these classes form the backbone of the framework.

2.1 `IndustrialObject.m`

An abstract base class inheriting directly from MATLAB's `handle`.

- **Properties:** `ID` (string identifier), `Type`, `Description`.
- **Methods:** Enforces the implementation of `getTags()`, `update(dt)`, and `reset()`.

2.2 `Equipment.m`

The workhorse abstract class inheriting from `IndustrialObject`. All process and electrical units inherit from this.

- **Standardized Structs:** `Parameters` (static config), `State` (dynamic internal variables), `Inputs` (values driven by external components), `Outputs` (values visible to external components), `Faults` (boolean flags).
- **Fault Management:** Provides `injectFault(name)` and `clearFault(name)` which interact with child classes to physically alter simulation behavior.
- **Status Tracking:** Tracks whether the equipment is "OK", "FAULTED", or "RUNNING".

Chapter 3

Process Equipment Library

Located in `library/process/`, these classes model thermodynamic and hydraulic physics.

3.1 ETank.m (Cylindrical Tank)

Simulates fluid accumulation using mass conservation.

- **Physics:** $dV/dt = Q_{in} - Q_{out} - Q_{leak}$. Level is derived as $h = V/(\pi \cdot r^2)$.
- **Faults:** **Leakage** (introduces a leak flow proportional to the current head/level), **SensorFailure** (sets the measured output level to NaN while leaving the physical volume intact).

3.2 EPipe.m (Pipeline)

Simulates pressure drop using simplified Darcy-Weisbach friction.

- **Physics:** $\Delta P = R_{friction} \cdot Q^2 + \rho \cdot g \cdot \Delta z$.
- **Faults:** **Blockage** (multiplies friction resistance by 100), **Leak** (diverts a percentage of incoming flow so it never reaches the output).

3.3 EPump.m (Centrifugal Pump)

Models both hydraulic performance and motor inertia.

- **Physics:** Motor speed approaches target via first-order lag ($\tau \cdot dN/dt = N_{target} - N$). The hydraulic head follows affinity laws: $H = H_{max} \cdot (N/N_{rated})^2 - k \cdot Q^2$. Power draw is proportional to the cube of the speed.
- **Faults:** **BearingFault** (drops efficiency, artificially increasing power consumption), **MotorFailure** (speed coasts to zero).

3.4 Valves: EValve.m & EControlValve.m

- **EValve:** Simple on/off valve with a configurable stroke time.
- **EControlValve:** Modulating valve accepting a 0-100% command. Simulates first-order actuator delay and three flow profiles: Linear, Equal-Percentage ($f(x) = R^{x-1}$), and Quick-Opening ($f(x) = \sqrt{x}$).
- **Faults:** **StuckOpen**, **StuckClosed**, **Stuck** (freezes at current position), **ActuatorFailure** (drives to a predefined fail-safe state).

3.5 Advanced Units: EMixer, EHeater, EFilter

- **EMixer:** Accepts two inflows with varying concentrations. Assumes perfect mixing and calculates the output concentration dynamically. Fault: **AgitatorFailure**.
- **EHeater:** Adds thermal energy (Q) to a flowing mass (m). $dT = Q/(m \cdot C_p)$. Fault: **ElementFailure**.
- **EFilter:** Tracks particulate accumulation over time, causing an exponential increase in pressure drop. Fault: **Rupture** (pressure drop collapses).

Chapter 4

Electrical & Actuation Library

Located in `library/electrical/`.

4.1 `EMotor.m` & `EVFD.m`

- **EMotor:** An independent motor model. Tracks spin-up time constants and cubic power laws. Faults: `Overheat`, `Tripped`.
- **EVFD:** Variable Frequency Drive. Intercepts a 0-100% setpoint and modulates the attached `EMotor`'s parameters to simulate variable speed operation. Fault: `DriveFault`.

4.2 `EConveyor.m`

Simulates material transport. Tracks load (kg/m) and speed (m/s).

- **Physics:** Transport delay and mass balance across the belt length.
- **Faults:** `BeltSlip` (speed halves), `MotorTrip`.

Chapter 5

Instrumentation Layer

Located in `library/instrumentation/`.

5.1 ETransmitter.m (Base Sensor)

A wrapper class that reads a specific property from a target `Equipment` and applies signal degradation.

- **Signal Chain:**

$$Measured = (True \times SpanDrift) + ZeroDrift + \mathcal{N}(0, \sigma^2)$$

The measured value is then clamped to sensor limits and converted to a 4-20mA electrical signal.

- **NAMUR NE43 Faults:**

- `SignalLoss`: Wire break. Current drops to 0mA, quality becomes `BAD_COMM`.
- `FrozenSignal`: ADC lockup. Signal holds the last value indefinitely (`UNCERTAIN_STALE`).
- `CalibrationLoss`: Massive 20% span error injection.

Specialized subclasses (`ELevelTransmitter`, `EFlowTransmitter`, `EPressureTransmitter`) automatically map to specific engineering units and target properties to reduce boilerplate code during instantiation.

Chapter 6

SCADA & System Architecture

Located in `library/historian/` and `library/system/`.

6.1 ETagDatabase.m (Plant Server)

Acts as an OPC UA / SCADA server. All equipment must be registered here. The `scanAll()` method loops through all devices, executes their `getTags()` methods, and vertically concatenates the results into a `MasterTagTable`. It maintains a `containers.Map` for $O(1)$ fast tag lookups.

6.2 EHistorian.m (Time-Series Logger)

Binds to the `ETagDatabase` and records tag histories.

- **Memory Management:** To prevent MATLAB memory fragmentation during long loops, it utilizes pre-allocated arrays (`BufferSize`). When the buffer is full, it dynamically doubles in size.
- **Data Retrieval:** `getHistory("TAG")` slices the active data and returns a native MATLAB `timetable`, ready for plotting or Machine Learning.

6.3 EFactory.m & ESimulationEngine.m

- **EConnection:** Physically maps the output property of a Source to the input property of a Target, routing flow data automatically.
- **EFactory:** A container holding all Equipment, Connections, Database, and Historian. Its `update(dt)` method enforces the order of execution: Connections $\rightarrow$ Equipment Physics $\rightarrow$ Database Scan $\rightarrow$ Historian Log.
- **ESimulationEngine:** The execution loop manager that runs the factory for a designated time duration given a discrete time step Δt .

Chapter 7

Testing and Validation

The framework was developed using Test-Driven Development (TDD) via the `matlab.unittest` framework.

7.1 Test Coverage

A total of **92 automated tests** were written across multiple suites:

- **ETankTest.m (15 tests):** Validates volume conservation, overflow limits, alarms (LL/L/H/HH), leakage dynamics, and fault resets.
- **EPumpTest.m, EPipeTest.m, EValveTest.m:** Verifies Darcy-Weisbach curves, affinity laws, and valve transition times.
- **ETransmitterTest.m (7 tests):** Tests Gaussian noise std-dev validation, signal limits, and NAMUR fault states.
- **ETagDatabaseTest.m & EHistorianTest.m:** Validates that tag arrays grow correctly, and timetables retain exact values mapped to timestamps.

Final Result: 92/92 tests passing with zero failures.

Chapter 8

Examples & Use Cases

Five main examples reside in the `examples/` directory.

8.1 TankExample.m

This script focuses purely on the hydraulics of static storage. It runs a single tank through a sequence of filling and draining cycles to validate mass conservation and verifies the triggering of Level (L/LL/H/HH) alarms.

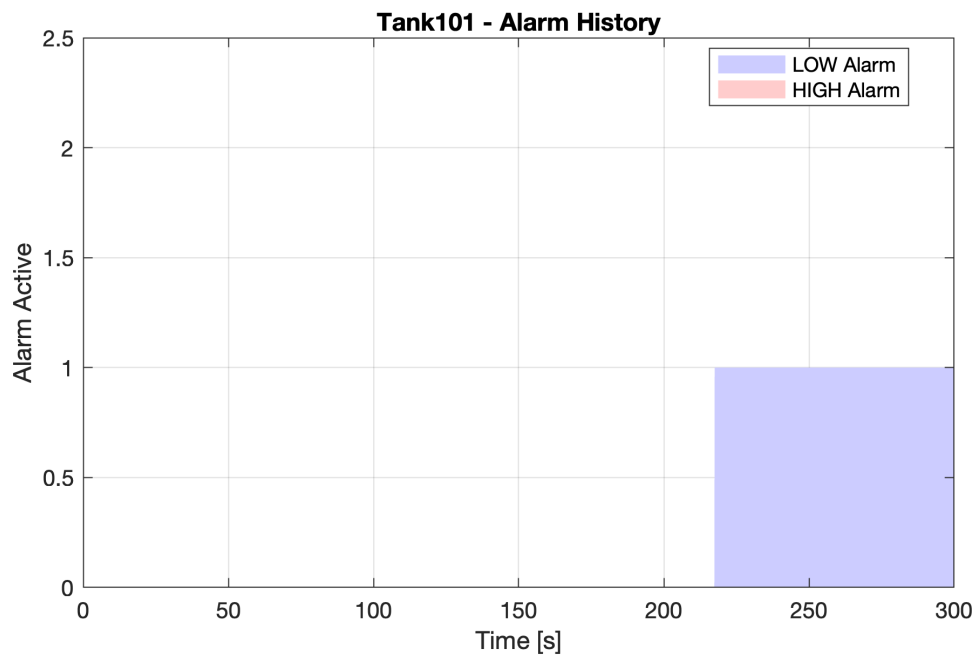


Figure 8.1: Tank example showcasing fluid levels, faults, and alarm thresholds.

8.2 PumpTankExample.m (Early Phase)

This example introduces dynamic flow transfer. It links a centrifugal pump to a downstream control valve and tank, allowing the user to observe how changes in valve position affect flow resistance and the resulting tank level.

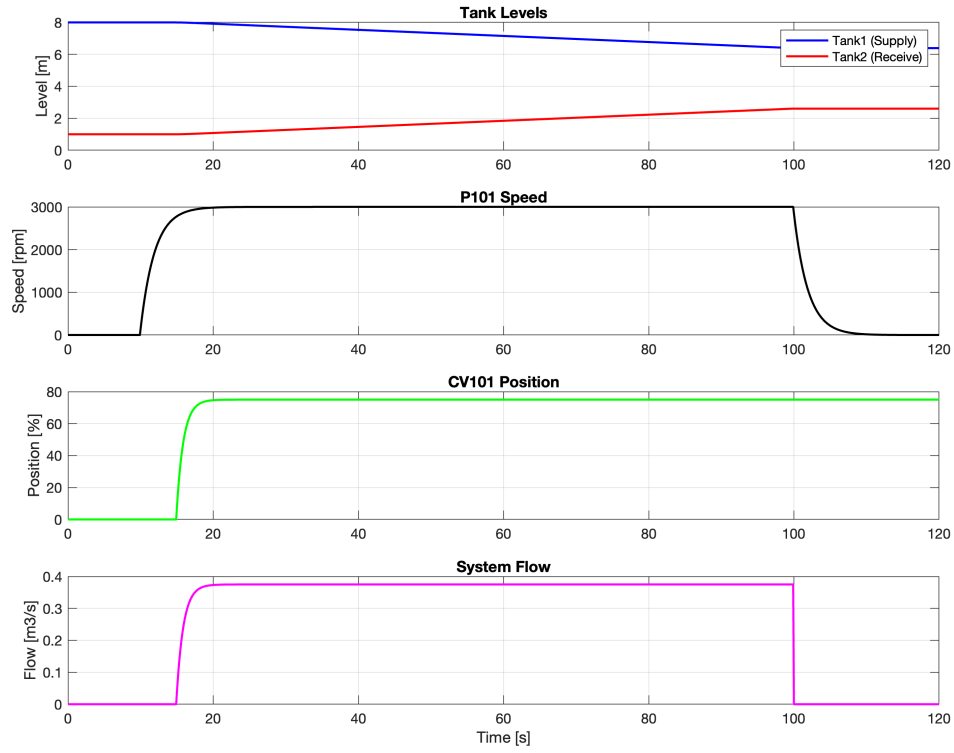


Figure 8.2: Pump, Tank, and Valve integration example.

8.3 HistorianExample.m

A backend-focused benchmark script. It evaluates the performance of the `ETagDatabase` mapping system and ensures that the `EHistorian` can log thousands of data points without memory fragmentation, outputting a clean MATLAB timetable.

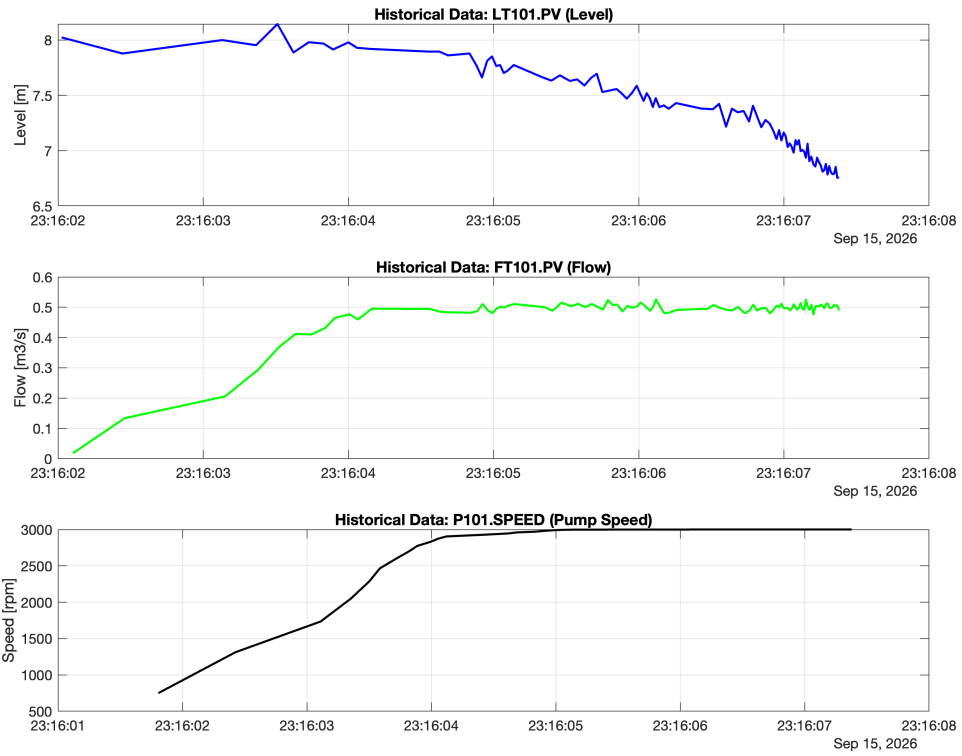


Figure 8.3: Historian time-series generation and native MATLAB plotting.

8.4 InstrumentationExample.m (Mid Phase)

Highlights the sensory degradation layer. It runs a side-by-side comparison of the actual physics engine state versus the noisy, 4-20mA clamped signal output by the `ETransmitter`, including a mid-simulation cable break fault.

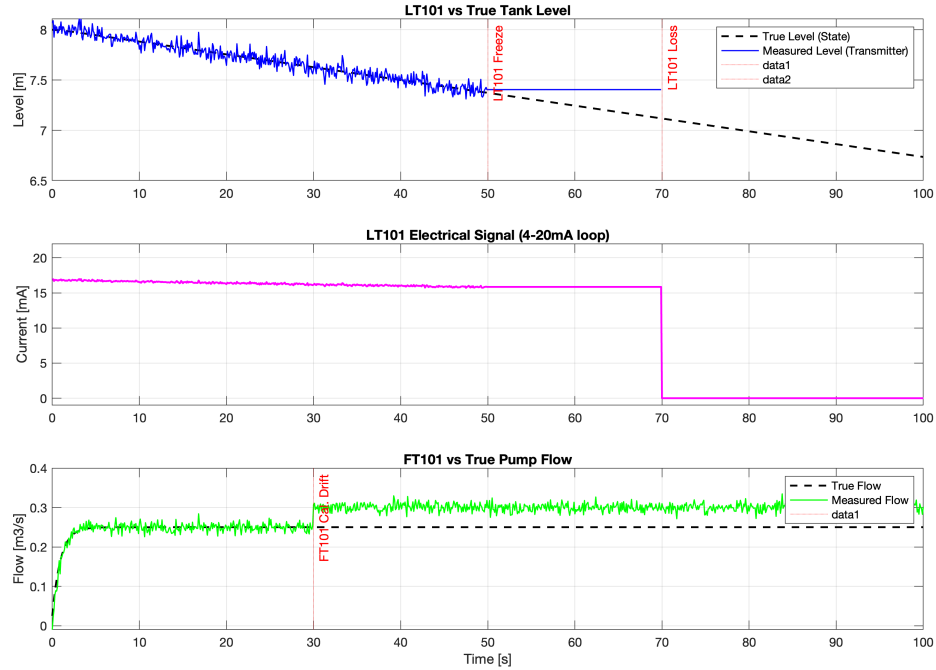


Figure 8.4: Instrumentation Example showing Ground Truth vs. Measured signals with Noise and Faults.

8.5 FullFactoryExample.m (Final Phase)

The culmination of the project. It builds an entire plant:

- Two source tanks pumped by VFD-controlled motors through control valves.
- A Mixer combining the two flows with different concentrations.
- A Heater applying thermal energy.
- A Filter accumulating pressure drop.
- A final product tank.

The script injects a *Heater Element Failure* and a *Filter Rupture* dynamically. It uses the `ESimulationEngine` to run the plant and extracts data natively from the `EHistorian` for final visualization.

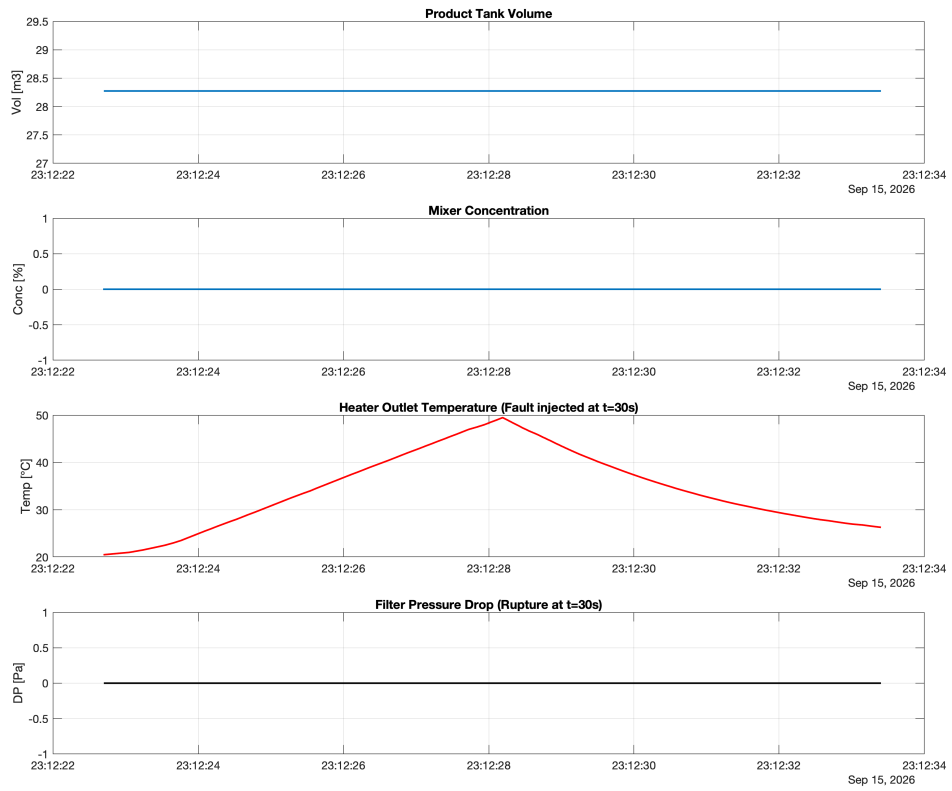


Figure 8.5: Full Factory Simulation showcasing Historian Tag extraction for Volume, Concentration, Temperature, and Pressure Drop over time.

Chapter 9

User Manual: How to Build Your Own Plant

9.1 Step 1: Instantiating the Environment

To begin, you must create the `EFactory` which automatically provisions the Tag Database and Historian.

```
1 factory = EFactory();
```

9.2 Step 2: Declaring Equipment

Create physical process units using their constructors. You can override default parameters such as dimensions or rated limits.

```
1 % Create a 10m high tank
2 TankA = ETank("ID", "TK101", "Diameter", 5, "Height", 10, "InitialLevel", 8);
3
4 % Create a motor and its VFD
5 MotorA = EMotor("ID", "MTR101", "RatedSpeed", 1500, "RatedPower", 15);
6 VFD_A = EVFD("ID", "VFD101", "Motor", MotorA);
```

9.3 Step 3: Attaching Instrumentation

To read variables into the SCADA system, attach Transmitters to the process equipment.

```
1 % Monitor TankA's level with a maximum range of 10m
2 LT_A = ELevelTransmitter("ID", "LT101", "Target", TankA, "RangeMax", 10);
```

9.4 Step 4: Registration

Add all instantiated components to the factory. This step ensures that the Tag Database registers them for polling.

```
1 factory.add(TankA);
2 factory.add(MotorA);
3 factory.add(VFD_A);
4 factory.add(LT_A);
```

9.5 Step 5: Running the Simulation

Initialize the `ESimulationEngine` with the factory and a discrete time step (e.g., $dt = 0.5$ seconds). In a custom loop, you can script hydraulic connections (mapping flows between tanks and pumps), inject faults, and call `factory.update(dt)`.

```
1 engine = ESimulationEngine(factory, 0.5);
2 for k = 1:100
3     % Inject a fault at step 50
4     if k == 50
5         LT_A.injectFault('SignalLoss');
6     end
7
8     % Step the physics and log tags
9     factory.update(0.5);
10 end
```

9.6 Step 6: Extracting Data

Once the loop finishes, extract the generated timetables natively.

```
1 % Extract native MATLAB timetable
2 ttLevel = factory.Historian.getHistory("LT101.PV");
3
4 % Plot time-series data natively
5 plot(ttLevel.Time, ttLevel.Value);
```

Chapter 10

Conclusion

To summarize, this project demonstrates the feasibility of combining rigorous mathematical physics with modern object-oriented software design in MATLAB. Rather than relying on simple scripts, the Digital Twin architecture acts as a comprehensive sandbox. It enables engineers to test complex control loops, validate SCADA alarming strategies, and generate high-fidelity datasets for Machine Learning applications, all before any physical hardware is deployed.